

Τελική Εργασία Μαθήματος

«Σύγχρονα Θέματα Τεχνολογίας Λογισμικού - Λογισμικό για κινητές
Συσκευές»

Γιούριϊ Οσιπιάν Π22125

UnipiCityVibe

Πίνακας Περιεχομένων

1. [Εισαγωγή](#)
2. [Τεχνολογίες](#)
3. [Αρχιτεκτονική](#)
 - a. [Δομή Κώδικα](#)
 - b. [Βαση Δεδομένων](#)
4. [Λειτουργίες Εφαρμογής](#)
 - a. [Authendication](#)
 - b. [Εκδηλώσεις](#)
 - i. [Λίστα Εκδηλώσεων](#)
 - ii. [Χάρτης Εκδηλώσεων](#)
 - iii. [Notifications](#)
 - c. [Κρατήσεις](#)
 - d. [Ρυθμίσεις \(SharedPreferences\)](#)
 - e. [Πολυγλωσσικότητα](#)
5. [Εκτέλεση εφαρμογής](#)
6. [Στιγμιότυπα Οθόνης Εφαρμογής](#)

Εισαγωγή

Το **UnipiCityVibe** είναι μία **native** εφαρμογή για συσκευές **Android**. Στόχος της εφαρμογής είναι να προσφέρει στον χρήστη γρήγορη και εύκολη εύρεση εκδηλώσεων που πραγματοποιούνται στην περιοχή του. Η ειδοποίηση του χρήστη γίνεται μία φορά για κάθε εκδήλωση μέσω notifications. Επιπλέον, σε περίπτωση που θελήσει να δει τι συμβαίνει γύρω του, μπορεί να χρησιμοποιήσει την εφαρμογή για να αναζητήσει διαθέσιμες εκδηλώσεις στην περιοχή του.

Παρέχεται επίσης η δυνατότητα προβολής είτε μιας λίστας με κοντινές εκδηλώσεις είτε ενός χάρτη που εμφανίζει τα σημεία όπου πραγματοποιούνται οι κοντινές εκδηλώσεις

Τεχνολογίες

Κατηγορίες	Τεχνολογίες
IDE	Android-Studio 2025.3.4.7-1
Γλώσσα	JAVA 11 / minSDK 33 / targetSDK 36
Database	Firebase Realtime Database
Authentication	Firebase Authentication
Χάρτης	Google Maps
Build System	Gradle 8.13

Σημείωση:

- Για τη σωστή λειτουργία του χάρτη απαιτείται η ύπαρξη ενός **API Key**, το οποίο πρέπει να δημιουργηθεί από αυτόν που θα κάνει compile την εφαρμογή. Η δημιουργία του πραγματοποιείται μέσω του **Google Cloud Console**. Αφού δημιουργηθεί, θα πρέπει να προστεθεί στο αρχείο **local.properties** στο πεδίο **MAPS_API_KEY**
- Επιπλέον, για σκοπούς **debugging** χρησιμοποιήθηκε η **open-source εφαρμογή LeakCanary**. Το αντίστοιχο dependency είναι προς το παρόν σχολιασμένο (**commented**). Σε περίπτωση που επιθυμείται η χρήση του, αρκεί να γίνει **uncomment** στο αρχείο **app/build.gradle.kts**

Αρχιτεκτονική

Δομή Κώδικα

Η εφαρμογή χωρίζεται σε **Data, Service, Managers, Utils** και **UI**

- **Data:**
 - **Singleton** κλάσεις που υπάρχουν καθ' όλη τη διάρκεια εκτέλεσης της εφαρμογής. Οι κλάσεις αυτές είναι υπεύθυνες για τη διασύνδεση της εφαρμογής με το **Backend**

- **Local:** Δεδομένα που αποθηκεύονται τοπικά στη συσκευή, όπως οι ρυθμίσεις της εφαρμογής
- **Service: Singleton** κλάσεις που υπάρχουν καθ' όλη τη διάρκεια εκτέλεσης της εφαρμογής. Οι κλάσεις αυτές υλοποιούν την επιχειρηματική λογική της εφαρμογής
- **Managers:** Βοηθητικές κλάσεις με σκοπό την απλοποίηση της χρήσης της κύριας επιχειρηματικής λογικής
- **Utils:** Γενικές στατικές βοηθητικές κλάσεις που παρέχουν κοινές λειτουργίες
- **UI:** Οτιδήποτε σχετίζεται με την εμφάνιση της εφαρμογής και την αλληλεπίδραση του χρήστη με αυτή

Για μια πιο ολοκληρωμένη εικόνα της αρχιτεκτονικής της εφαρμογής, υπάρχουν τα αντίστοιχα **Class Diagrams** στον φάκελο **doc/***

Βάση Δεδομένων

Στη βάση δεδομένων υπάρχει αυστηρός διαχωρισμός των δεδομένων στις συλλογές **Events**, **Tickets** και **Users**

- **Events:** Είναι Read-Only, καθώς υπάρχει ένα προκαθορισμένο σύνολο εκδηλώσεων (12 εκδηλώσεις)
 - **Model:** event_id -> {title, description, time, price, latitude, longitude}
- **Tickets:** Καθώς δεν υπάρχει σύστημα πληρωμών, ο χρήστης μπορεί να προσθέτει εισιτήρια μόνο για τον δικό του λογαριασμό και μόνο για διαθέσιμες εκδηλώσεις
 - **Model:** user_id -> {ticket_id -> {event_id, user_id, timestamp}}
- **Users:** Καθώς δεν υπάρχει σύστημα πληρωμών, ο χρήστης μπορεί να προσθέτει εισιτήρια μόνο για τον δικό του λογαριασμό και μόνο για διαθέσιμες εκδηλώσεις
 - **Model:** user_id -> {name, last_name, email}

Για περισσότερες λεπτομέρειες σχετικά με τους κανόνες πρόσβασης και ασφάλειας της βάσης δεδομένων, υπάρχει το αρχείο **doc/rules_firebase.json**, το οποίο περιέχει τους αντίστοιχους κανόνες (**Firestore Rules**)

Λειτουργίες Εφαρμογής

Authentication

Για τη σύνδεση του χρήστη με την υπηρεσία **Firestore** υπάρχει η κλάση **UserAuth.java**. Η κλάση αυτή παρέχει τις λειτουργίες **createUser**, **signIn**, **signOut** και **deleteUser** μέσω του **Firestore Authentication**

Επιπλέον, υπάρχει η κλάση **UserDB.java**, η οποία, όταν ο χρήστης είναι συνδεδεμένος, παρέχει τις λειτουργίες **addUser** και **deleteUser** στο **Firestore Realtime Database**. Σε αντίθεση με την **UserAuth.java**, η κλάση αυτή δεν διαχειρίζεται τη σύνδεση ή αποσύνδεση του χρήστη, αλλά αποκλειστικά την προσθήκη και διαγραφή των δεδομένων του στη βάση δεδομένων

Η κλάση που αναλαμβάνει την ορθή και συντονισμένη χρήση των παραπάνω κλάσεων είναι η **AuthService.java**. Με αυτόν τον τρόπο, το **UI** αρκεί να επικοινωνεί με την **AuthService.java**, η οποία αναλαμβάνει την εκτέλεση των απαραίτητων ενεργειών ώστε να υλοποιηθούν οι λειτουργίες που σχετίζονται με το **authentication**

Εκδηλώσεις

Για τη λήψη των εκδηλώσεων υπάρχει η κλάση **EventDB.java**, η οποία είναι υπεύθυνη για την ανάκτηση των δεδομένων από τη βάση. Πάνω από αυτήν λειτουργεί η κλάση **EventService.java**, η οποία υλοποιεί ασύγχρονη λήψη δεδομένων μέσω της **EventDB.java** και επιστρέφει τις εκδηλώσεις που βρίσκονται κοντά στον χρήστη όταν αυτές ζητηθούν

Σημείωση: Η κλάση **EventService.java** δεν γνωρίζει τη θέση του χρήστη. Για τον λόγο αυτό, η θέση πρέπει να μεταφέρεται σε αυτή μέσω της κλάσης **LocationService.java**, ώστε να είναι δυνατός ο υπολογισμός και η επιστροφή των κοντινών εκδηλώσεων

Ο χρήστης μπορεί να δει τις εκδηλώσεις με τρεις διαφορετικούς τρόπους:

Λίστα Εκδηλώσεων

Στην κλάση **EventListAroundFragment.java**, για την εμφάνιση των εκδηλώσεων που βρίσκονται γύρω από τον χρήστη σε μορφή λίστας, γίνεται χρήση της κλάσης **EventService.java**

Χάρτης Εκδηλώσεων

Στην κλάση **MapFragment.java**, για την εμφάνιση του χάρτη χρησιμοποιείται το **Google Maps** με το αντίστοιχο **Google API Key**

Εκτός από τον χάρτη και τη θέση του χρήστη, εμφανίζονται και σημεία (**points**) που αντιστοιχούν στις εκδηλώσεις που βρίσκονται κοντά του. Για την ανάκτηση και εμφάνιση των εκδηλώσεων στον χάρτη γίνεται χρήση της κλάσης **EventService.java**

Notifications

Για τη διαχείριση των **notifications** υπάρχει η βοηθητική κλάση **EventNotificationManager.java**. Το αντικείμενό της δημιουργείται μέσα στην κλάση **UserActivity.java** και καταστρέφεται μαζί με αυτή

Η συγκεκριμένη προσέγγιση ακολουθείται επειδή οι ειδοποιήσεις αποτελούν λειτουργία που εκτελείται στο παρασκήνιο μόνο όταν ο χρήστης είναι συνδεδεμένος στην εφαρμογή. Για να επιτευχθεί αυτή η λειτουργία. Έγινε χρήση του **handler**, ο οποίος σε τακτά χρονικά διαστήματα ελέγχει τον χρόνο και, μέσω των κλάσεων **EventService.java** και **NotificationService.java**, εμφανίζει ειδοποιήσεις για τις εκδηλώσεις που βρίσκονται κοντά στον χρήστη που δεν έχουν ξανά εμφανιστεί

Σημείωση: Μέσα από το **NotificationService.java** υλοποιείται η λειτουργία των ειδοποιήσεων στην εφαρμογή

Κρατήσεις

Εκτός από τη δυνατότητα προβολής των εκδηλώσεων, ο χρήστης μπορεί επίσης να πραγματοποιεί κρατήσεις, καθώς και να βλέπει τις εκδηλώσεις για τις οποίες έχει ήδη κάνει κράτηση. Η λειτουργικότητα αυτή παρέχεται μέσω της κλάσης **TicketService.java**

Η υλοποίηση βασίζεται στον συνδυασμό των κλάσεων **EventDB.java** και **TicketDB.java**. Η **EventDB.java** περιέχει τα δεδομένα των εκδηλώσεων, ενώ η **TicketDB.java** αποθηκεύει τις πληροφορίες των κρατήσεων, δηλαδή ποιοι χρήστες έχουν κάνει κράτηση σε ποιες εκδηλώσεις, καθώς και τη χρονική στιγμή κατά την οποία πραγματοποιήθηκε η κράτηση

Ρυθμίσεις (SharedPreferences)

Υπάρχει ξεχωριστό παράθυρο για τις ρυθμίσεις του χρήστη. Οι ρυθμίσεις χωρίζονται σε τρεις κατηγορίες:

- **User Settings:** Στις ρυθμίσεις χρήστη, ο χρήστης μπορεί να δει τα στοιχεία του λογαριασμού του και, εφόσον το επιθυμεί, να διαγράψει τον λογαριασμό του.
- **Permission Settings:** Στις ρυθμίσεις δικαιωμάτων, και εφόσον έχουν δοθεί οι αντίστοιχες άδειες από το λειτουργικό σύστημα, ο χρήστης μπορεί να διαχειριστεί τις άδειες που σχετίζονται με τη λειτουργία της εφαρμογής:
 - Άδεια ειδοποιήσεων (Notifications)
 - Άδεια πρόσβασης στην τοποθεσία (Location)
- **General Settings:** Στις γενικές ρυθμίσεις, ο χρήστης έχει τη δυνατότητα να προσαρμόσει την εμφάνιση της εφαρμογής:
 - Αλλαγή γραμματοσειράς
 - Ενεργοποίηση ή απενεργοποίηση της σκοτεινής λειτουργίας (**Dark Mode**)

Για την αποθήκευση των ρυθμίσεων χρησιμοποιείται το **SharedPreferences** μέσω της κλάσης **AppSettings.java**

Σημείωση: Για να λειτουργήσουν σωστά όλες οι ρυθμίσεις. Μετά από κάθε αλλαγή γίνεται **recreate** του **activity**

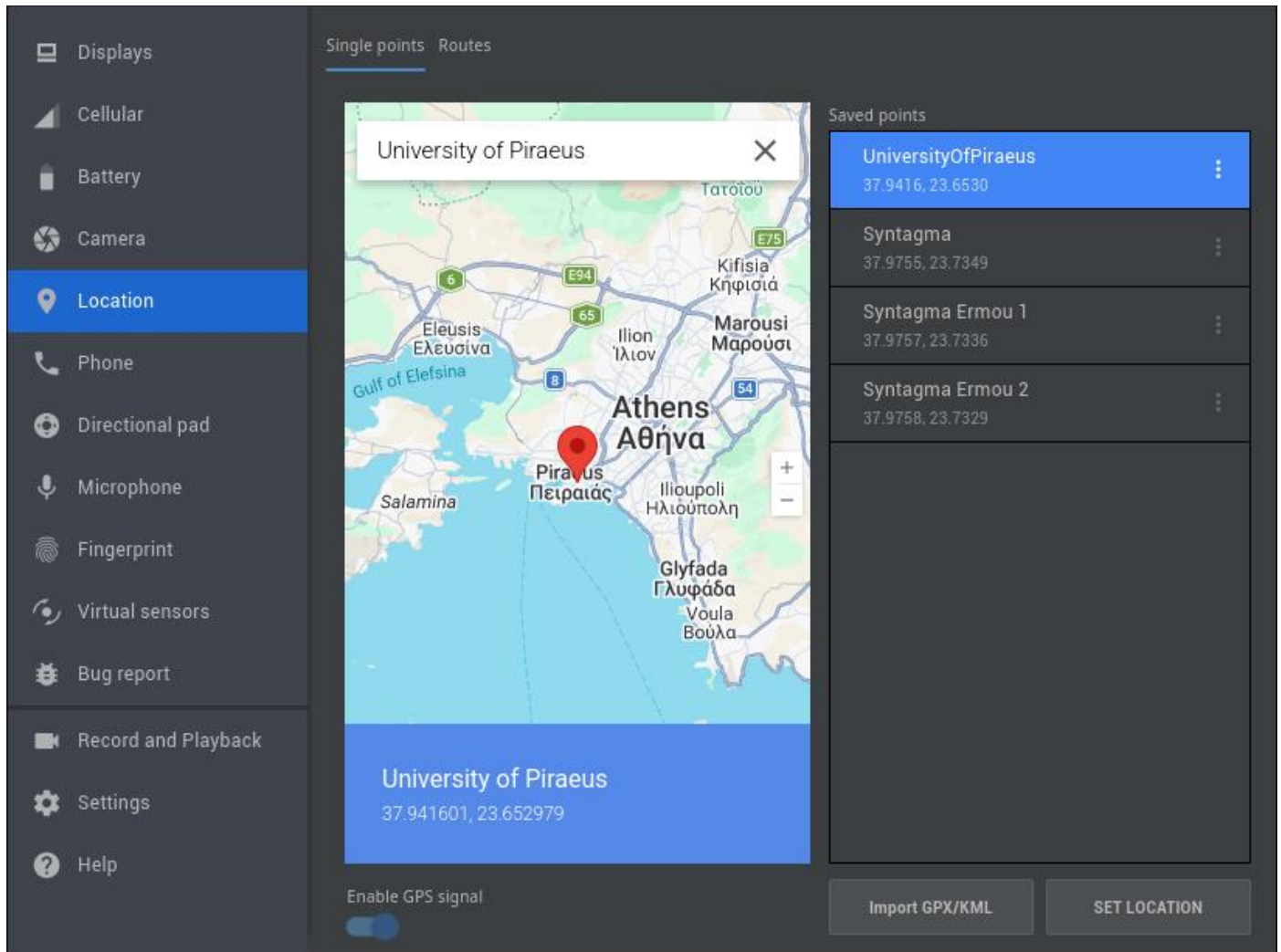
Πολυγλωσσικότητα

Η εφαρμογή διαθέτει τρεις γλώσσες. **Αγγλικά, Ελληνικά και Ρωσικά**. Σε περίπτωση που θελήσει ο χρήστης να αλλάξει την γλώσσα. Όπως και τις ρυθμίσεις θα γίνει μέσα από το **AppSettings.java**

Εκτέλεση Εφαρμογής

Για την σωστή εκτέλεση της εφαρμογής και δοκιμής της θα πρέπει να γίνουν πρώτα κάποια βήματα:

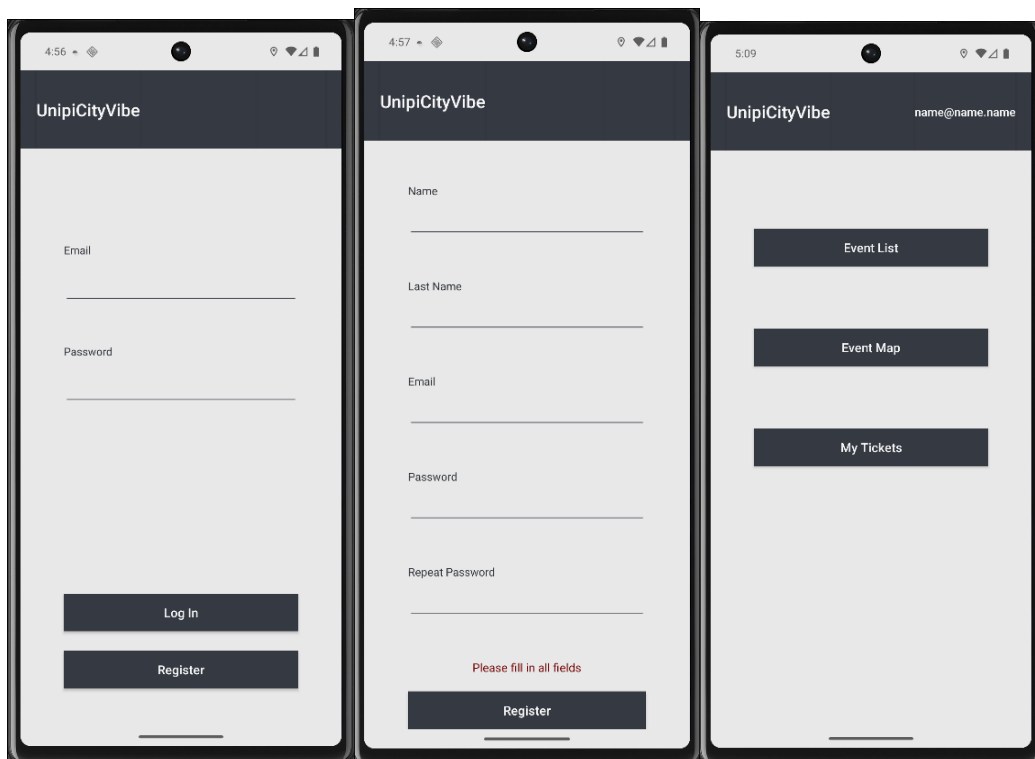
- Για την σωστή λειτουργία του χάρτη θα πρέπει να υπάρχει **Google Maps API Key** από το **Google Cloud Console**. Αυτό το κλειδί θα πρέπει να τοποθετηθεί στο αρχείο **local.properties** στο πεδίο **MAPS_API_KEY**
- Για σωστές δοκιμές τις εφαρμογής θα χρειαστεί να ορίσετε στο **emulator** τοποθεσίες του χρήστη όπου γύρω του θα υπάρχουν εκδηλώσεις. Τοποθεσίες που χρησιμοποιήθηκαν κατά τις δοκιμές της εφαρμογής



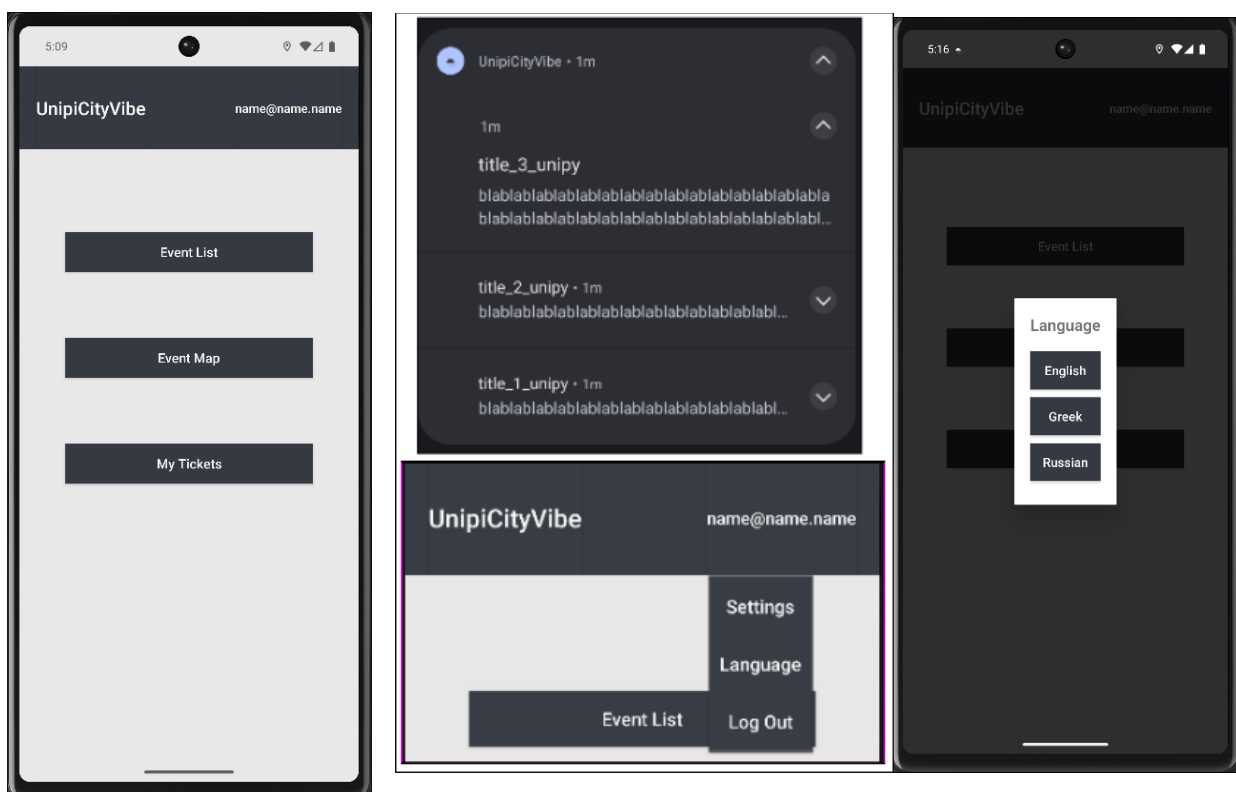
Στιγμιότυπα οθόνης εφαρμογής

Εδώ υπάρχουν στιγμιότυπα τα οποία δείχνουν την λειτουργικότητα της εφαρμογής

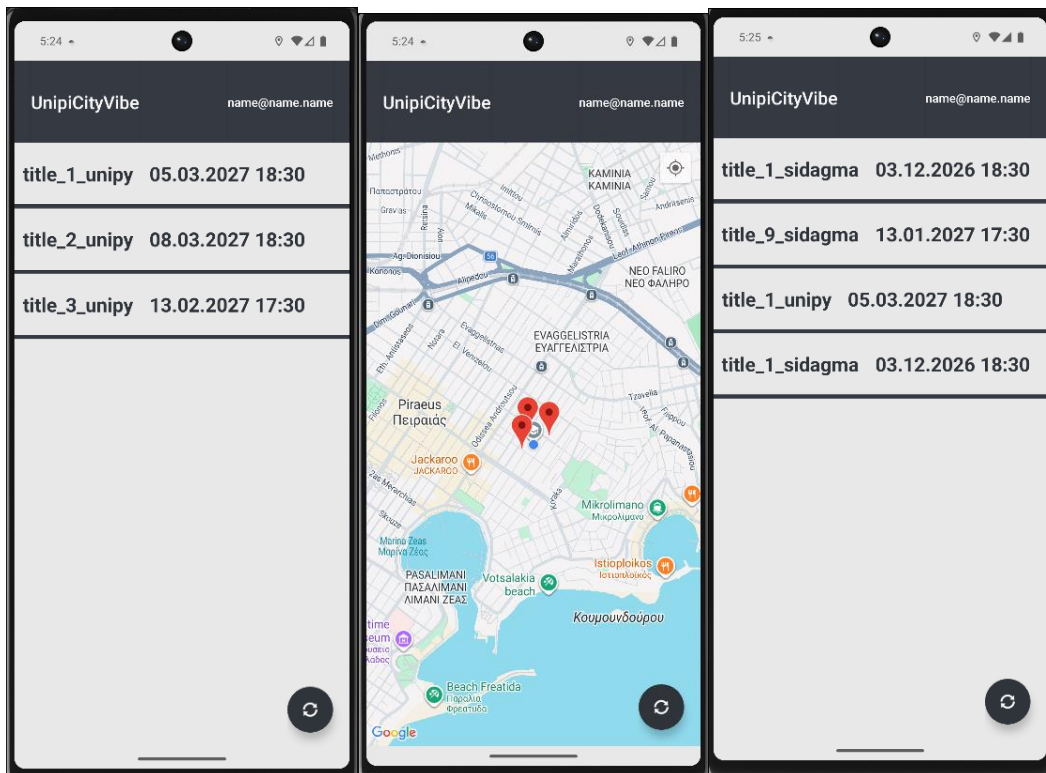
Σύνδεση/Εγγραφή του Χρήστη:



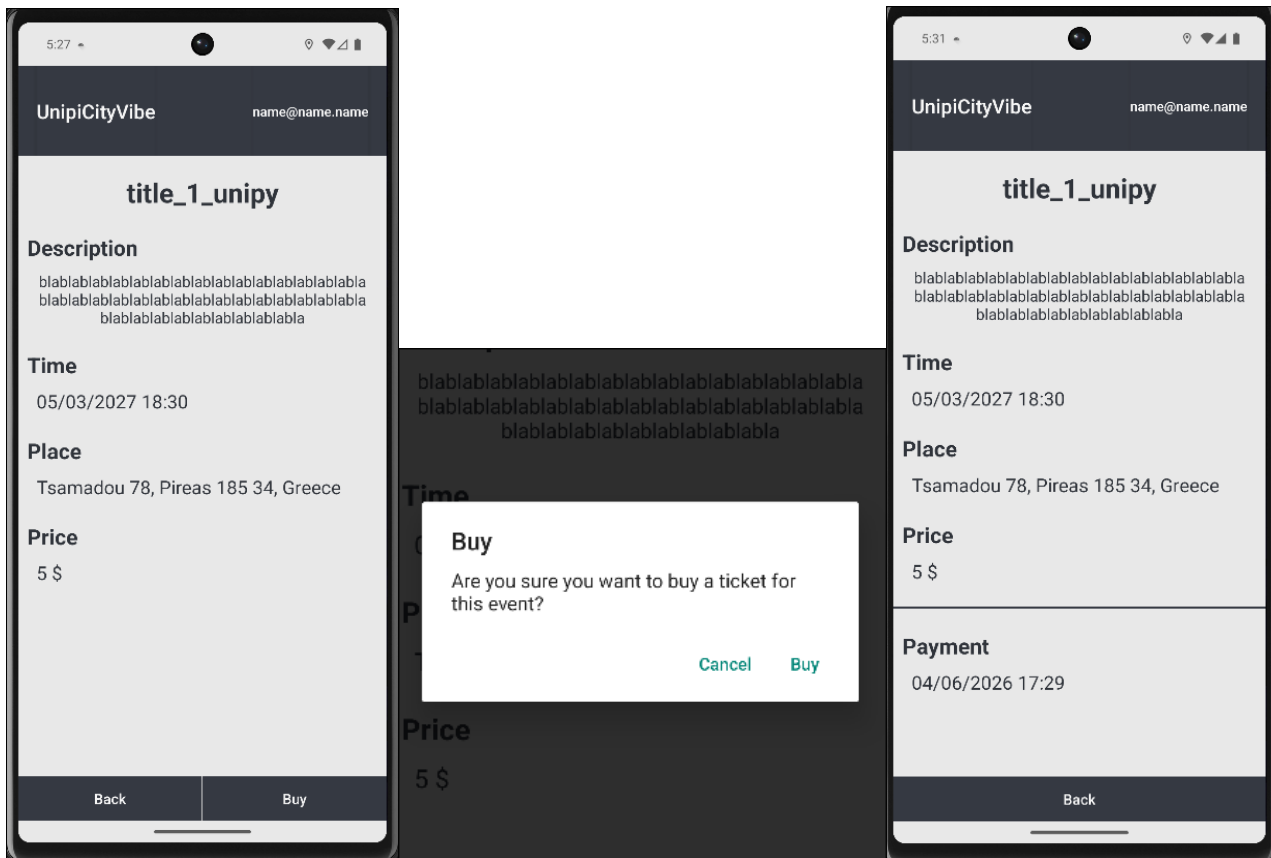
Κύρια Οθόνη, Ειδοποιήσεις, Πάνω Μπάρα της εφαρμογής και Αλλαγή Γλώσσας:



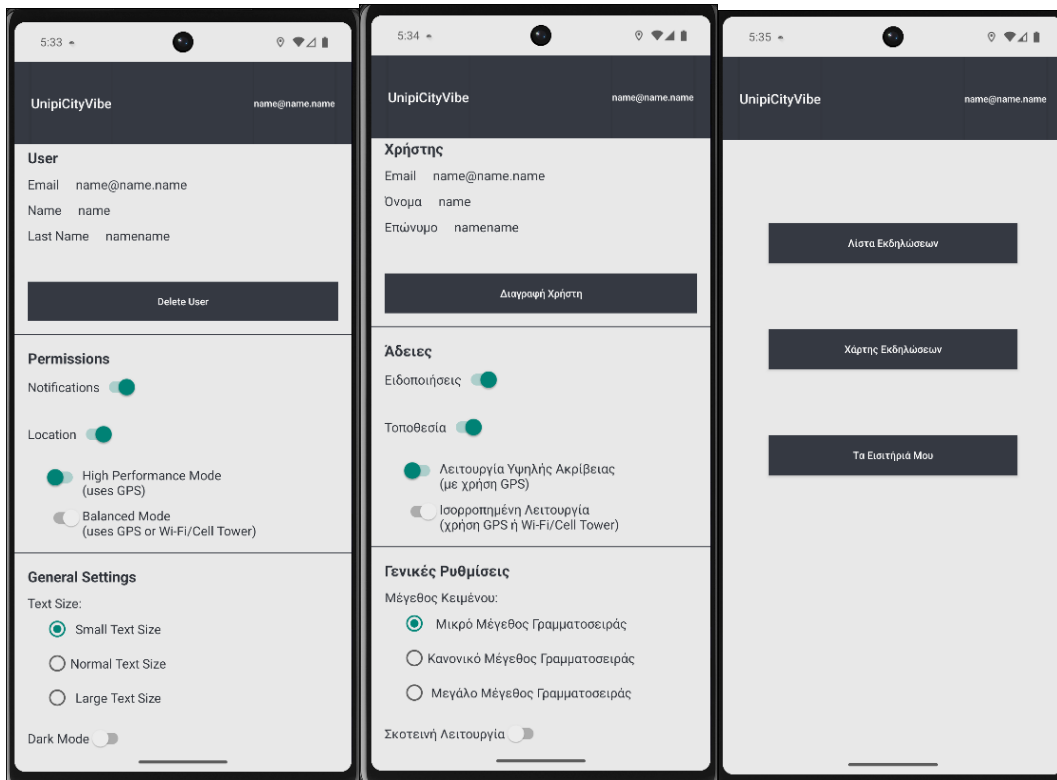
Λίστα Εκδηλώσεων, Χάρτης Εκδηλώσεων και Τα Εισιτήρια Μου:



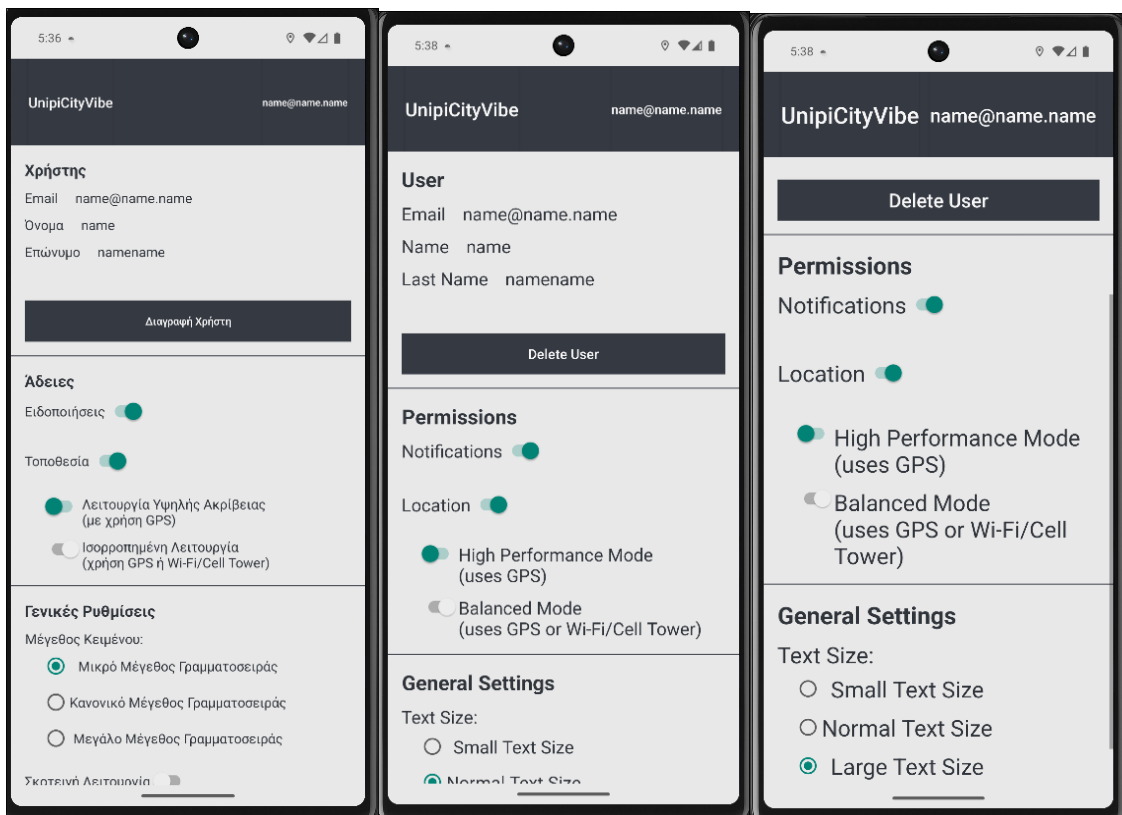
Πληροφορίες Για Την Εκδήλωση, Αγορά Εισιτηρίου και Πληροφορίες Για Το Εισιτήριο:



Ρυθμίσεις και Παράδειγμα Αλλαγής Γλώσσας:



Παράδειγμα Αλλαγή Μέγεθους Γραμματοσειράς:



Παράδειγμα Σκοτεινής Λειτουργίας:

